

# Rapport : Résolution du problème NP-Difficile de Kemeny à l'aide de méthodes de décomposition

Goales Saphiya et Teibi Zahra

Numéro étudiant : 21233538 et 21211965

# Table des matières

|          |                                                     |           |
|----------|-----------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                 | <b>2</b>  |
| <b>2</b> | <b>Préliminaires / Notations</b>                    | <b>2</b>  |
| <b>3</b> | <b>Méthodes de décomposition</b>                    | <b>4</b>  |
| 3.1      | La règle de majorité des 3/4 . . . . .              | 4         |
| 3.2      | Le critère de Condorcet étendu . . . . .            | 6         |
| <b>4</b> | <b>Résolution des sous-instances</b>                | <b>7</b>  |
| 4.1      | Programmation Dynamique . . . . .                   | 7         |
| 4.2      | Programmation linéaire . . . . .                    | 9         |
| <b>5</b> | <b>Expérimentations</b>                             | <b>10</b> |
| 5.1      | Instances réelles . . . . .                         | 10        |
| 5.1.1    | Comparaison des méthodes de réduction . . . . .     | 11        |
| 5.1.2    | Analyse des temps de résolution . . . . .           | 12        |
| 5.2      | Génération d'instances . . . . .                    | 12        |
| 5.2.1    | Comparaison des méthodes de décomposition . . . . . | 13        |
| 5.2.2    | Evaluation de la programmation linéaire . . . . .   | 13        |
| <b>6</b> | <b>Conclusion</b>                                   | <b>13</b> |

# 1 Introduction

On cherche à trouver un classement sur les candidats qui respecte les préférences individuelles. Parmi les méthodes existantes, l'agrégation de classement constitue un problème fondamental, avec des applications allant de l'apprentissage automatique à la recherche web [1].

Parmi les règles de vote, la règle de Kemeny établit un classement de consensus qui minimise la distance totale de Kendall-tau définie comme le nombre total de désaccords (paires inversées) entre le consensus et l'ensemble des préférences.

D'un point de vue théorique, la règle de Kemeny possède des propriétés robustes, telles que le fait de satisfaire à la fois :

- le *critère de Condorcet*, c'est à dire que si un candidat, confronté à chaque autre candidat, est toujours préféré par la majorité des votants (vainqueur de Condorcet), alors il sera classé premier dans le classement de Kemeny [2, 3].
- l'axiome de *renforcement*, qui stipule que si l'on considère deux groupes distincts de votants pour lesquels le classement de consensus est le même, alors ce même résultat doit être obtenu lorsque l'on combine les votes des deux groupes [3].
- la *neutralité* qui garantit que si deux candidats échangent leurs positions dans les préférences de tous les votants, ils doivent également échanger leurs positions dans le classement final. Ainsi, tous les candidats sont traités de manière égale [3].

C'est la seule règle qui respecte ces 3 propriétés, malgré leur caractère naturel.

**Problématique :** cette solidité théorique est confrontée à une difficulté pratique majeure : sa complexité. En effet, le calcul d'un classement de Kemeny est un problème NP-difficile [4, 5], même lorsque le nombre de votants est fixé et petit, pour  $n = 4$ , par exemple [5].

Afin de répondre à cette problématique, des méthodes de décompositions ont été proposées. Ces méthodes décomposent l'instance originale en de plus petits sous-problèmes indépendants qu'il faudra ensuite résoudre.

Ce projet vise à implémenter des approches permettant de résoudre le problème de Kemeny de manière exacte et efficace, même sur des instances de grande taille. Ainsi l'objectif est de comparer les méthodes de résolution (programmation dynamique et programmation linéaire) ainsi que les méthodes de décomposition (le critère de Condorcet étendu [6] et la règle de majorité des 3/4 [4]). Afin d'obtenir des résultats d'évaluation réalistes, nous évaluerons ces méthodes sur des instances réelles stockées dans la bibliothèque Preflib [7].

## 2 Préliminaires / Notations

On considère un ensemble fini  $\mathcal{C} = \{1, \dots, m\}$  de  $m$  candidats et un ensemble  $\mathcal{N} = \{1, \dots, n\}$  de  $n$  votants. On note  $\mathcal{L}(\mathcal{C})$  l'ensemble des ordres linéaires, aussi appelés classements des candidats de  $\mathcal{C}$ . Chaque votant  $i \in \mathcal{N}$  donne un classement  $P_i \in \mathcal{L}(\mathcal{C})$ , appelé préférence de  $i$ . On note  $P = \{P_1, P_2, \dots, P_n\}$ , l'ensemble contenant les préférences des votants, aussi appelé profil de préférences.

La règle de Kemeny consiste à chercher un ordre linéaire  $P^* \in \mathcal{L}(\mathcal{C})$  qui minimise la somme des désaccords total avec le profil de préférence. Ce désaccord est mesuré par le score de Kemeny [4], noté  $\Delta$ , c'est à dire la somme des distances de Kendall-Tau entre un classement  $R$  et un profil de préférences  $P$  :

$$\Delta(R, P) = \sum_{i \in \mathcal{N}} D_\tau(R, P_i).$$

où  $D_\tau(P_i, P^*)$  est la distance de Kendall-Tau entre deux classements, définie de la manière suivante :

$$D_\tau(P_i, P^*) = \sum_{(a,b) \in \mathcal{C}^{\ominus}} \mathbf{1}_{\{a \succ b \text{ dans } P_i \text{ et } b \succ a \text{ dans } P^*\}}.$$

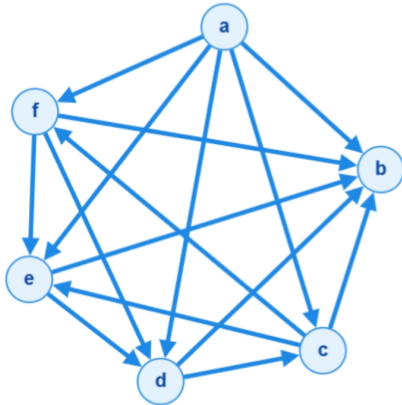
**Exemple 1.** Si l'on considère une instance avec  $m = 6$  candidats et  $n = 8$  votants avec les préférences suivantes :

- $P_1 : a \succ c \succ f \succ e \succ d \succ b$
- $P_2 : a \succ c \succ f \succ e \succ d \succ b$
- $P_3 : a \succ c \succ e \succ f \succ d \succ b$
- $P_4 : a \succ d \succ c \succ f \succ e \succ b$
- $P_5 : a \succ d \succ c \succ f \succ e \succ b$
- $P_6 : a \succ b \succ d \succ c \succ f \succ e$
- $P_7 : a \succ b \succ f \succ e \succ d \succ c$
- $P_8 : a \succ b \succ f \succ e \succ d \succ c$ .

Pour déterminer le classement de Kemeny  $P^*$ , nous calculons la somme des distances de Kendall-Tau entre  $P^*$  et les préférences des votants. Dans le cas où  $P^*$  est égal à  $a \succ c \succ e \succ f \succ d \succ b$ , on a  $D_\tau(P_1, P^*) = 1$  (seule la paire  $(e, f)$  est inversée).

**Définition 1.** Le graphe de majorité strict  $G^{P, \mathcal{C}} = (\mathcal{C}, \mathcal{A})$  est un graphe orienté dont les noeuds correspondent aux candidats, et un arc entre deux candidats  $a$  et  $b$  indique que le candidat  $a$  est préféré par plus de 50% des votants au candidat  $b$ .

**Exemple 2.** On considère le profil de préférences de l'exemple 1.



Le graphe de majorité stricte construit est celui ci-contre. On observe que le candidat  $a$  est un vainqueur de Condorcet.

**Définition 2.** Etant donné un ensemble de candidats  $\mathcal{C}$  et un profil de préférence  $P$  sur les candidats de  $\mathcal{C}$ , on définit la matrice de préférence  $M^{P, \mathcal{C}} \in \mathbb{N}^{m \times m}$  telle que  $M_{i,j}^{P, \mathcal{C}}$  représente le nombre de votants préférant le candidat  $i$  au candidat  $j$ .

**Exemple 3.** Pour l'instance de l'exemple 1, la matrice de préférences  $Q$  est la suivante :

$$M^{P, \mathcal{C}} = \begin{pmatrix} & \textcolor{red}{a} & \textcolor{green}{b} & \textcolor{blue}{c} & \textcolor{orange}{d} & \textcolor{violet}{e} & \textcolor{magenta}{f} \\ \textcolor{red}{a} & 0 & 8 & 8 & 8 & 8 & 8 \\ \textcolor{green}{b} & 0 & 0 & 3 & 3 & 3 & 3 \\ \textcolor{blue}{c} & 0 & 5 & 0 & 3 & 6 & 6 \\ \textcolor{orange}{d} & 0 & 5 & 5 & 0 & 3 & 3 \\ \textcolor{violet}{e} & 0 & 5 & 2 & 5 & 0 & 1 \\ \textcolor{magenta}{f} & 0 & 5 & 2 & 5 & 7 & 0 \end{pmatrix}$$

### 3 Méthodes de décomposition

La réduction de données vise à identifier et à fixer des relations d'ordre entre candidats qui sont garanties d'apparaître dans toute solution optimale. Cette réduction constitue une phase de prétraitement essentielle pour rendre la résolution d'instances de grande taille abordable.

#### 3.1 La règle de majorité des 3/4

La règle de majorité des 3/4 repose sur la notion de préférence forte et permet d'identifier des candidats dont la position relative peut être déterminée sans ambiguïté.

**Définition 3** (Betzler, Bredereck et Niedermeier [4]). *Soit  $a$  et  $b$  deux candidats. Nous disons que  $a$  a une préférence forte sur  $b$  ( $a \succ_{3/4} b$ ) si  $a$  est préféré à  $b$  par au moins 3/4 des votants.*

**Définition 4** (Betzler, Bredereck et Niedermeier [4]). *Un candidat propre  $a$  est défini tel que pour tout autre candidat  $b$  il existe une préférence forte entre  $a$  et  $b$  ( $a \succ_{3/4} b$  ou  $b \succ_{3/4} a$ ).*

**Lemme 1** (Betzler, Bredereck et Niedermeier [4]). *Soit  $a \in \mathcal{C}$  un candidat propre pour la règle de  $s$ -majorité et  $b \in \mathcal{C} \setminus \{a\}$  avec  $3/4 \leq s \leq 1$ . Si  $a \succ_s b$ , alors dans tout classement de Kemeny  $a$  est classé avant  $b$  ( $a \succ b$ ); si  $b \succ_s a$ , alors dans tout classement de Kemeny  $b$  est classé avant  $a$  ( $b \succ a$ ).*

La règle de majorité des 3/4 s'appuie sur le Lemme 1 pour ordonner les candidats propres. On connaît également l'ordre relatif entre chaque candidat propre et chaque autre candidat non propre.

Les candidats non propres nécessitent un traitement plus approfondi, car leur position relative reste incertaine même après application de la règle de majorité des 3/4. Les candidats non propres d'une même position constituent une sous-instance. Le classement final de Kemeny s'obtient en concaténant les classements résolus des sous-instances et des candidats propres [4].

L'algorithme de la règle des 3/4 identifie d'abord les candidats propres. Puis, il établit un ordre complet sur les candidats.

Afin d'implémenter cette règle de décomposition, on vérifie si le premier candidat est propre. Si le candidat est propre, on applique la règle de majorité des 3/4 sur les candidats classés avant et sur les candidats classés après. S'il ne l'est pas, on l'ignore et on passe au candidat suivant. Lorsqu'on sort de la boucle il ne reste que des candidats non propres. Ainsi on crée une sous-instance avec ces candidats puis on l'ajoute au classement.

---

**Algorithm 1** *est\_propre*

---

**Input :** *cand*, *candidats*  
**Output :** (*l\_avant*, *l\_apres*, *propre*)

*l\_avant*  $\leftarrow \emptyset$   
*l\_apres*  $\leftarrow \emptyset$   
*propre*  $\leftarrow \text{True}$

**for** *i*  $\in$  *candidats* **do**  
  **if** *i* = *cand* **then**  
    **continue**  
  **else if** *matPref*[*i*, *cand*]  $\geq \frac{3}{4} \cdot \text{nb\_votants}$  **then**  
    *l\_avant.add*(*i*)  
  **else if** *matPref*[*cand*, *i*]  $\geq \frac{3}{4} \cdot \text{nb\_votants}$  **then**  
    *l\_apres.add*(*i*)  
  **else**  
    *propre*  $\leftarrow \text{False}$   
  **end if**  
**end for**

**if** not *propre* **then**  
  **return** (*l\_avant*, *l\_apres*, *False*)  
**else**  
  **return** (*l\_avant*, *l\_apres*, *True*)  
**end if**

---

---

**Algorithm 2** *majorite\_trois\_quarts\_rec*

---

**Input :** *l\_candidats*, *classement*

*i*  $\leftarrow 0$   
**while** *i* < |*l\_candidats*| **do**  
  (*l\_avant*, *l\_apres*, *propre*)  $\leftarrow \text{est\_propre}(\text{candidats}[i], \text{candidats})$   
  **if** *propre* **then**  
    *majorite\_trois\_quarts\_rec*(*l\_avant*, *classement*)  
    Ajouter *l\_candidats*[*i*] à la fin de *classement*  
    *majorite\_trois\_quarts\_rec*(*l\_apres*, *classement*)  
    **return**  
  **else**  
    *i*  $\leftarrow i + 1$   
  **end if**  
**end while**  
Créer une sous-instance *nv\_inst* à partir de l'ensemble *l\_candidats*  
Ajouter *nv\_inst* à la fin de *classement*  
**return**

---

La création d'une sous-instance est en  $\mathcal{O}(1)$ . La vérification qu'un candidat soit propre se fait en  $\mathcal{O}(m)$ , car on compare ce candidat à tous les autres.

La boucle *while* est exécutée au plus  $m$  fois, ainsi le coût de chaque appel est  $\mathcal{O}(m^2)$ . Le nombre d'appels récursifs est borné par  $2m$ . Par conséquent, la règle décompose l'instance en  $\mathcal{O}(2m \times m^2) = \mathcal{O}(m^3)$ .

Dans notre cas, le facteur  $n$  (nombre de votants) disparaît car les comparaisons entre candidats sont effectuées à partir de la matrice de préférences, évitant ainsi de parcourir explicitement l'ensemble des votants.

**Exemple 4.** On considère le profil de préférences de l'exemple 1.

Les candidats propres sont : **a**.

Les candidats non propres sont : **b**, **c**, **d**, **e**, **f**.

On teste si le candidat **a** est propre. Comme **a** est propre, on applique la règle des 3/4 sur l'ensemble des candidats avant **a** ( $\{\}$ ) et après ( $\{\mathbf{b}, \mathbf{c}, \mathbf{d}, \mathbf{e}, \mathbf{f}\}$ ).

Pour l'ensemble vide, la solution est triviale.

On teste ensuite si **b** est propre. Il ne l'est pas, donc on le place en fin de liste et on applique la règle des 3/4 sur l'ensemble  $\{\mathbf{c}, \mathbf{d}, \mathbf{e}, \mathbf{f}, \mathbf{b}\}$ . On boucle sur cet ensemble car tous les candidats sont non propres. La variable essais conditionne l'arrêt et empêche une exécution infinie de la boucle. En effet, lorsque essais atteint le nombre de candidats, l'algorithme a fait le tour de tous les candidats ; il ne reste donc plus que des candidats non propres dans l'ensemble.

Ainsi, le classement optimal partiel est :  $\mathbf{a} \succ \{\mathbf{b}, \mathbf{c}, \mathbf{d}, \mathbf{e}, \mathbf{f}\}$

### 3.2 Le critère de Condorcet étendu

Le critère de Condorcet étendu [6] constitue une version plus général du critère de Condorcet, qui se concentre sur un unique vainqueur.

**Définition 5** (Truchon [6]). Soit  $C_\alpha$  et  $C_\beta$  deux ensembles de candidats. Nous disons que  $C_\alpha$  est préféré à  $C_\beta$  ( $C_\alpha \succ C_\beta$ ) si tout candidat  $a \in C_\alpha$  est préféré à tout candidat  $b \in C_\beta$  par au moins 50% des votants.

Le principe est le suivant : si un sous-ensemble de candidats  $C_\alpha$  est majoritairement préféré à son complémentaire  $C_\beta = C \setminus C_\alpha$ , alors  $C_\alpha$  se classe en tête de tout classement optimal. Ce principe peut être généralisé avec plusieurs sous-ensembles formant une partition de l'ensemble des candidats  $\mathcal{C}$ . Le classement de Kemeny sur  $\mathcal{C}$  est obtenu en juxtaposant les classements de Kemeny trouvés sur les sous-ensembles d'une partition.

L'algorithme du critère de Condorcet étendu consiste à construire le graphe de majorité stricte, puis à classer les composantes fortement connexes.

Dans un profil de préférences, il y a deux types de relations :

- les relations transitives : si une majorité préfère  $a$  à  $b$ ,  $b$  à  $c$  et  $a$  à  $c$ . Ces candidats forment nécessairement des composantes fortement connexe isolées dans le graphe, car il n'existe pas d'arc de  $c$  vers  $a$ . Ce raisonnement se généralise avec un plus grand nombre de candidats.
- les relations cycliques : ce cycle sera retranscrit dans le graphe de majorité.

Afin d'implémenter cet algorithme, nous avons utilisé une classe Graphe interne à la classe Instance. Un graphe est défini par une liste de noeuds et un dictionnaire d'adjacence. Cette classe permettra, par la suite, de créer le graphe de majorité d'une instance. Ce graphe peut être affichée grâce à la fonction `afficher_graphe`, qui utilise la bibliothèque `networkx` ; ce qui nous permet de vérifier les résultats visuellement.

Pour identifier et classer les composantes fortement connexes, nous avons implémenté l'algorithme de Tarjan [8] dans la fonction `condorcet_etendu`. Une composante fortement connexe de taille  $> 1$  représente un groupe incohérent, soit une sous-instance à résoudre. Par conséquent, pour chacune d'entre elles, on crée un objet de la classe Instance.

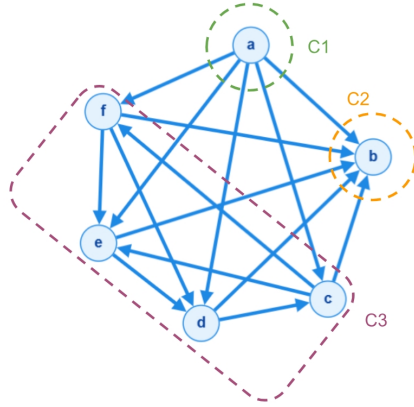
L'algorithme de Tarjan trouve les composantes fortement connexes d'un graphe orienté en un seul parcours en profondeur. Il attribue à chaque sommet un numéro d'ordre (indice de découverte) et une valeur lowlink qui représente le plus petit numéro accessible depuis ce sommet. Lorsqu'un sommet a son lowlink égal à son numéro d'ordre, il est la racine d'une composante fortement connexe. On dépile alors tous les sommets jusqu'à ce sommet racine : ils forment une composante fortement connexe.

La création d'une sous-instance est en  $\mathcal{O}(1)$  et la construction du graphe de majorité stricte est en  $\mathcal{O}(m^2)$ . La complexité de l'algorithme de Tarjan est  $\mathcal{O}(m + m^2) = \mathcal{O}(m^2)$ .

Le nombre d'appels récursifs (au total) est borné par  $m$ . Par conséquent, la règle décompose l'instance en  $\mathcal{O}(m^3)$ .

De même, le facteur  $n$  (nombre de votants) disparaît car les comparaisons entre candidats sont effectuées à partir de la matrice de préférences, évitant ainsi de parcourir explicitement l'ensemble des votants.

**Exemple 5.** On considère le profil de préférences de l'exemple 1.



Le graphe de majorité stricte construit est celui ci-contre. On ordonne les composantes fortement connexes de la manière suivante :  $\mathcal{C}_1 \succ \mathcal{C}_3 \succ \mathcal{C}_2$ .

*Remarque :* le critère de Condorcet étendu réduit plus l'instance que la règle de majorité des 3/4.

Si un candidat est propre selon la règle des 3/4, il domine ou est dominé par chaque autre candidat avec un seuil d'au moins 75%, ce qui implique qu'il existe un arc orienté dans un sens ou l'autre entre lui et tout autre candidat dans le graphe de majorité (seuil à 50%). Ainsi, ce candidat ne peut appartenir à aucun cycle car la règle de majorité est transitive. Si un candidat propre  $c$  est dominé par un candidat  $d1$  et domine un candidat  $d2$ ,  $d1$  domine également  $d2$ . Il n'existe pas de chemin entre  $d2$  et  $d1$ .

Donc le critère de Condorcet étendu réduit au moins autant que la règle de majorité des 3/4.

## 4 Résolution des sous-instances

Les règles de décomposition transforment le problème initial NP-difficile en un ensemble de sous-problèmes indépendants de tailles beaucoup plus petites. Ces sous-problèmes doivent ensuite être résolus.

### 4.1 Programmation Dynamique

L'algorithme de programmation dynamique calcule, pour chaque sous-ensemble possible de candidats  $\mathcal{C}' \subseteq \mathcal{C}$ , le classement partiel qui minimise le score de Kemeny.

En effet, si l'on retire le candidat qui est en tête d'un classement optimal, l'ordre restant pour les autres candidats est lui aussi optimal pour ce sous-ensemble réduit. Donc, le classement de Kemeny peut être obtenu en sélectionnant le bon candidat en tête avec le classement de Kemeny pour les candidats restants.

$$A[\mathcal{C}_A] = \min_{c \in \mathcal{C}_A} \{A[\mathcal{C}_A \setminus \{c\}] + \text{cout}(c, \mathcal{C}_A)\}$$

où  $A[\mathcal{C}_A]$  représente le classement de Kemeny retourné par l'algorithme et  $\text{cout}(c, \mathcal{C}_A)$  le coût de placer le candidat  $c$  en tête.

**Théorème** (Fischer, Hudry et Niedermeier [5]). L'agrégation de classement de Kemeny peut être résolue en temps  $\mathcal{O}(2^m \cdot m^2 \cdot n)$ .

**Décomposition récursive [9] :**



**Cas de base :** pour les sous-ensembles  $\mathcal{C}'$  de taille 1, le calcul d'un classement de Kemeny est trivial. Il y a un seul candidat  $c$  et le score de Kemeny associé est de 0.

**Étape récurrente :** pour les sous-ensembles  $\mathcal{C}'$  de taille  $|\mathcal{C}'| > 1$ , l'algorithme teste itérativement chaque candidat  $c \in \mathcal{C}'$  comme étant celui placé en première position du classement local.

**Calcul du score :** le score total pour  $c$  est :  $\text{score}(c, \mathcal{C}') = \text{cout}(c, \mathcal{C}') + \text{score}(\mathcal{C}'')$ . Le  $\text{score}(\mathcal{C}'')$  est le score optimal déjà calculé pour l'ensemble des candidats restants  $\mathcal{C}''$  et le  $\text{cout}(c, \mathcal{C}')$  est le coût engendré par le placement de  $c$  en tête du classement  $\mathcal{C}'$  (nombre de désaccords sur les paires de candidats).

**Optimisation :** le candidat  $c$  qui minimise le score total, est donc placé en première position dans le classement optimal pour ce sous-ensemble  $\mathcal{C}'$ .

En procédant ainsi, l'algorithme assure que le score final  $\text{score}(\mathcal{C})$  est le score de Kemeny optimal pour l'ensemble complet  $\mathcal{C}$ , et le chemin suivi pour l'obtenir reconstitue le classement de Kemeny optimal.

Pour implémenter cet algorithme, nous avons écrit une fonction itérative, *resolution\_dyn*. Elle retourne un dictionnaire associant à chaque sous-ensemble de candidats un couple (candidat en tête, score). Pour chaque taille  $t > 1$  et pour chaque combinaison de candidats, nous testons chaque élément en première position du classement afin de ne conserver dans le dictionnaire que celui qui minimise le score. Dans le cas particulier où  $t=1$ , l'unique candidat est directement enregistré.

---

**Algorithm 3** *resolution\_dyn*

---

**Input :** *inst*

**Output :** *c\_opti* (dictionnaire : sous-ensemble  $\rightarrow$  (tête, score))

*c\_opti*  $\leftarrow \{\}$

**for**  $t = 1, \dots, \text{inst.nb\_candidats}$  **do**

**for**  $c \in \text{combinaison de taille } t \text{ des candidats}$  **do**

**if**  $t = 1$  **then**

*c\_opti*[ $c$ ]  $\leftarrow (c[0], 0)$

**else**

*score\_min*  $\leftarrow +\infty$

*cand\_en\_tete*  $\leftarrow c[0]$

**for**  $\text{cand} \in c$  **do**

*reste*  $\leftarrow c \setminus \{\text{cand}\}$

*score\_reste*  $\leftarrow \text{c\_opti}[\text{reste}][1]$

*cout\_cand*  $\leftarrow \text{cout}(\text{cand}, \text{reste}, \text{inst.matPref})$

**if**  $\text{score\_reste} + \text{cout\_cand} < \text{score\_min}$  **then**

*score\_min*  $\leftarrow \text{score\_reste} + \text{cout\_cand}$

*cand\_en\_tete*  $\leftarrow \text{cand}$

**end if**

**end for**

*c\_opti*[ $c$ ]  $\leftarrow (\text{cand\_en\_tete}, \text{score\_min})$

**end if**

**end for**

**end for**

**return** *c\_opti*

---

Ce dictionnaire permet ensuite de reconstruire le classement final via la fonction *reconstruction\_classement\_PDyn*. Cette dernière extrait successivement les candidats de tête pour aboutir à la liste ordonnée finale.

Le coût de placer un candidat en tête est  $\mathcal{O}(m)$  pour chaque candidat ; soit  $\mathcal{O}(m^2)$  au total. On calcule ce coût pour chaque sous-ensemble de taille  $k$  ( $2^m$  sous-ensembles). Par conséquent, la complexité de l'algorithme est  $\mathcal{O}(2^m \times m^2) = \mathcal{O}(m^2 \times 2^m)$ .

**Exemple 6.** *Application de la Programmation Dynamique*

Soit l'instance réduite par le critère de Condorcet étendue dans l'exemple 5. Soit dict un dictionnaire vide avec pour clé l'ensemble des candidats ordonnés et pour valeur le couple (candidat en tête, score).

Pour chaque sous-ensemble de taille 1 ( $\{\mathbf{c}\}$ ,  $\{\mathbf{d}\}$ ,  $\{\mathbf{e}\}$ ,  $\{\mathbf{f}\}$ ), on ajoute le couple ( $\{\text{candidat}\}$ , ( $\{\text{candidat}\}, 0$ )).

Sous-ensembles de taille 2 : On calcule le score de Kemeny optimal pour le sous-ensemble. Par exemple, pour le sous-ensemble  $\{\mathbf{c}, \mathbf{d}\}$  :

- $\mathbf{c} \succ \mathbf{d}$  engendre un coût de 5,
- $\mathbf{d} \succ \mathbf{c}$  engendre un coût de 3.

Le score minimal est donc 3 pour ce sous ensemble. On ajoute dans dict, le couple ( $\{\mathbf{c}, \mathbf{d}\}$ , ( $\{\mathbf{d}\}, 3$ )). On fait de même pour tous les autres.

Sous-ensembles de taille 3 : On calcule le score de Kemeny optimal pour le sous-ensemble, à partir des scores des sous-ensembles de taille inférieure. Par exemple, pour le sous-ensemble  $\{\mathbf{c}, \mathbf{d}, \mathbf{e}\}$  :

- $\mathbf{c} \succ \{\mathbf{d}, \mathbf{e}\}$  engendre un coût de  $7 + 3 = 10$ ,
- $\mathbf{d} \succ \{\mathbf{c}, \mathbf{e}\}$  engendre un coût de  $8 + 2 = 10$ ,
- $\mathbf{e} \succ \{\mathbf{c}, \mathbf{d}\}$  engendre un coût de  $9 + 3 = 12$ .

De même, on met à jour dict avec le nouveau couple ( $\{\mathbf{c}, \mathbf{d}, \mathbf{e}\}$ , ( $\{\mathbf{c}\}, 10$ )) (car  $\mathbf{c}$  est le premier candidat testé, et on met à jour le candidat en tête si et seulement le score est strictement inférieur). On fait de même pour tous les autres.

Sous-ensembles de taille 4 : De manière identique, on obtient :

- $\mathbf{c} \succ \{\mathbf{d}, \mathbf{e}, \mathbf{f}\}$  engendre un coût de  $9 + 7 = 16$ ,
- $\mathbf{d} \succ \{\mathbf{c}, \mathbf{e}, \mathbf{f}\}$  engendre un coût de  $13 + 5 = 18$ ,
- $\mathbf{e} \succ \{\mathbf{c}, \mathbf{d}, \mathbf{f}\}$  engendre un coût de  $16 + 10 = 26$ .
- $\mathbf{f} \succ \{\mathbf{c}, \mathbf{d}, \mathbf{e}\}$  engendre un coût de  $10 + 10 = 20$ .

On met à jour dict avec le nouveau couple ( $\{\mathbf{c}, \mathbf{d}, \mathbf{e}, \mathbf{f}\}$ , ( $\{\mathbf{c}\}, 16$ )).

Le score de Kemeny minimal est de 16 et correspond au cas où le candidat  $\mathbf{c}$  est placé avant les candidats  $\mathbf{d}$ ,  $\mathbf{e}$  et  $\mathbf{f}$  dans le classement final. De plus, le classement optimal pour  $\{\mathbf{d}, \mathbf{e}, \mathbf{f}\}$  est  $\mathbf{f} \succ \{\mathbf{d}, \mathbf{e}\}$  et pour  $\{\mathbf{d}, \mathbf{e}\}$ ,  $\mathbf{e} \succ \mathbf{d}$ . Ainsi, le classement final est  $\mathbf{c} \succ \mathbf{f} \succ \mathbf{e} \succ \mathbf{d}$ .

## 4.2 Programmation linéaire

L'agrégation de classement de Kemeny nécessite que le classement final soit un ordre strict. Ce problème peut donc être modélisé par le programme linéaire en nombres entiers suivant :

$$\min \sum_{a,b} M_{a,b}^{P,C} \cdot x_{ba} + M_{b,a}^{P,C} \cdot x_{ab} \quad (1)$$

$$\text{s.t. } x_{ab} + x_{ba} = 1 \quad \forall a \in \mathcal{C}, \forall b \in \mathcal{C} \setminus \{a\} \quad (2)$$

$$x_{ab} + x_{bc} + x_{ca} \geq 1 \quad \forall a \in \mathcal{C}, \forall b \in \mathcal{C} \setminus \{a\}, \forall c \in \mathcal{C} \setminus \{a, b\} \quad (3)$$

$$x_{ab} \in \{0, 1\} \quad (4)$$

Ce programme linéaire vise à minimiser le score total de Kemeny et retourne une solution du problème d'agrégation de Kemeny [4].

Le modèle utilise les variables binaires  $x_{ab} \in \{0, 1\}$  qui valent 1 si le candidat  $a$  est classé avant le candidat  $b$  dans le classement de Kemeny.

Les contraintes suivantes garantissent que les solutions  $x_{ab}$  représentent un ordre linéaire valide :

- Contraintes d'ordre strict (contrainte (2)) : cette contrainte assure que pour toute paire de candidats  $(a, b)$ , on ne peut pas avoir à la fois  $a \succ b$  et  $b \succ a$  dans le classement final.
- Contraintes de transitivité (contrainte (3)) : cette contrainte est essentielle car elle évite l'existence de cycle. En effet, pour chaque triplet de candidats distincts  $(a, b, c)$ , si  $a \succ b$  et  $b \succ c$  alors on a forcément  $a \succ c$ .

Nous avons donc implémenté une fonction *resolution\_pl* qui, à l'aide de Gurobi, résout ce programme linéaire. Pour ce faire, nous avons créé un modèle, en utilisant la matrice de préférences stockée dans l'instance (attribut *matPref*) pour définir les coefficients de la fonction objectif. Nous avons ensuite implémentée la fonction *reconstruction\_classement\_PL* qui, à partir du modèle résolu, renvoie le classement correspondant sous la forme d'une liste, ou lève une exception si aucune solution n'a été trouvée.

Bien que la taille de ce programme soit polynomiale :  $m^2$  variables et  $m^3$  contraintes, le problème d'agrégation de classement de Kemeny reste NP-difficile.

## 5 Expérimentations

Afin d'évaluer les performances des algorithmes implémentés, nous avons défini deux protocoles de tests basés sur des métriques distinctes : le temps d'exécution global et la taille de la plus grande sous-instance obtenue après réduction (*eval\_reduction*). La fonction *eval\_temps* permet de comparer le temps de résolution d'une instance selon différentes configurations : sans réduction et avec chaque méthode de décomposition.

Nos expérimentations portent sur deux types de jeux de données : des instances réelles issues de la bibliothèque *Preflib* [7] et des instances synthétiques générées via le modèle de *Mallows* [10].

### 5.1 Instances réelles

La bibliothèque Preflib est une bibliothèque de référence pour la recherche sur les préférences. Nous avons récupéré l'ensemble des jeux de données sur le dépôt GitHub de PrefLib : <https://github.com/PrefLib/PrefLib-Data/tree/main/datasets>. Ces instances couvrent des domaines variés, comme des résultats d'élections réelles ou des classements de films (Netflix) et sont stockées dans des fichiers au format *.soc* (*strict ordinal collections*).

Pour intégrer ces données à notre projet, la classe *Instance* dispose d'une méthode de lecture s'appuyant sur la bibliothèque *preflibtools.instances*. Le processus de traitement se déroule en deux étapes. D'abord, on parcourt le fichier *.soc* pour extraire le nombre de candidats, le nombre de votants ainsi que les ordres linéaires exprimés. Nous utilisons ensuite notre fonction *comptage* pour transformer ces votes en une matrice de préférences  $M^{P,C}$ .

Cette matrice constitue la structure de données centrale pour tous nos algorithmes de résolution ainsi que pour nos méthodes de décomposition, car elle résume toute l'information nécessaire au calcul du score de Kemeny sans avoir à reparcourir l'ensemble des votes initiaux.

### 5.1.1 Comparaison des méthodes de réduction

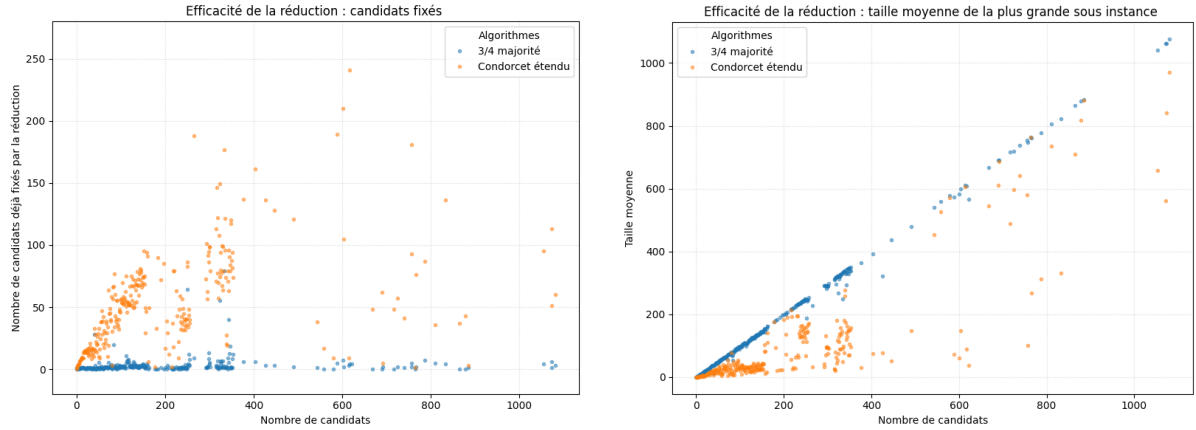


FIGURE 1 – Efficacité des réductions sur les instances de PrefLib

Pour confirmer notre intuition que le critère de Condorcet étendu réduit d’avantage que la règle de majorité des  $3/4$ , nous avons évalué, pour chaque instance, le nombre de candidats dont la position est connue après réduction et la taille de la plus grande sous-instance.

Les courbes ci dessus représentent la moyenne de ces métriques sur les instances regroupées par nombre de candidats.

Nous observons que, quelle que soit la taille de l’instance, la règle de majorité des  $3/4$  fixe très peu de candidats. La taille de la plus grande sous-instance croît proportionnellement au nombre de candidats : la courbe est pratiquement une droite de pente 1. Cela signifie que la réduction est négligeable : pour chaque instance, il subsiste une sous-instance presque aussi grande que l’instance initiale. L’efficacité de cette règle reste donc très limitée, puisqu’elle ne parvient pas à réduire significativement la taille des instances.

Au contraire, le critère de Condorcet étendu apparaît nettement plus efficace. Le nombre de candidats fixés augmente globalement avec la taille de l’instance jusqu’à environ 600 candidats, puis devient plus irrégulier. Ce comportement s’explique naturellement : plus une instance contient de candidats, plus il existe de possibilités d’en fixer. Ainsi, on ne peut pas conclure que le critère de Condorcet étendu devient systématiquement plus performant lorsque le nombre de candidats est très élevé. Nous observons également que la taille de la plus grande sous-instance augmente avec le nombre de candidats. Cependant, cette sous-instance reste toujours nettement plus petite que l’instance initiale, ce qui confirme que la réduction obtenue est réellement efficace.

Les points irréguliers observés au-delà de 400 candidats peuvent être attribués aux particularités de certaines instances. En effet, même lorsqu’une instance comporte un grand nombre de candidats, si les préférences des votants sont proches, la réduction peut être très efficace.

Néanmoins, malgré ces variations, le critère de Condorcet étendu demeure largement plus efficace que la règle de majorité des  $3/4$ .

### 5.1.2 Analyse des temps de résolution

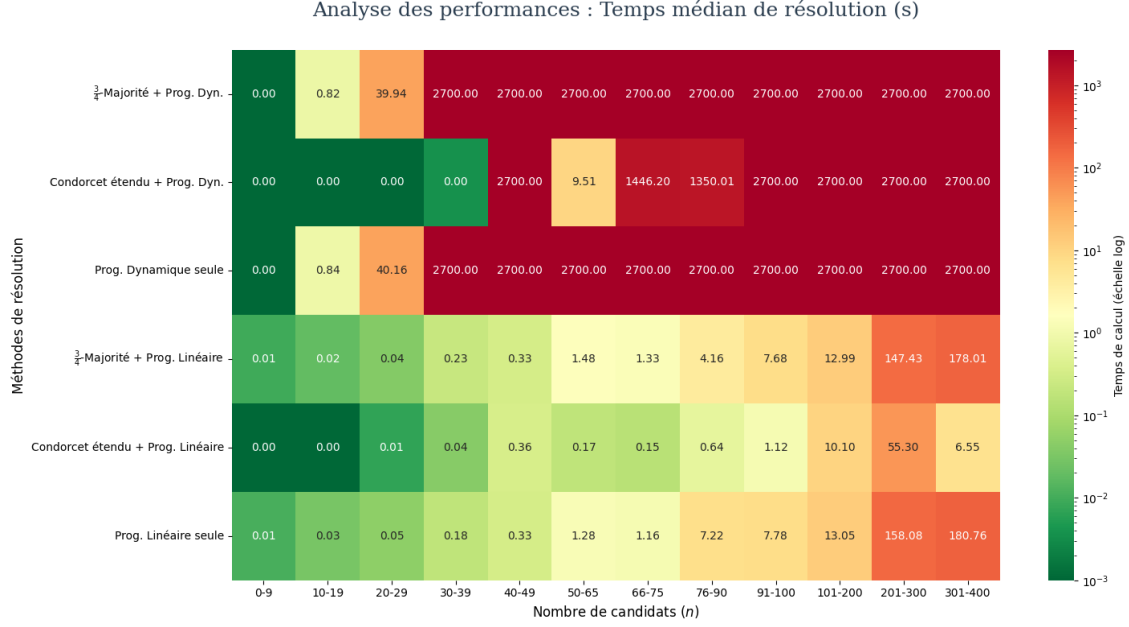


FIGURE 2 – Temps médian de résolution avec et sans réduction.

...à faire

## 5.2 Génération d'instances

Afin de tester la robustesse de nos algorithmes sur des données contrôlées, nous avons complété nos tests par des instances synthétiques générées selon le modèle de Mallows [10]. Ce modèle probabiliste permet de simuler des profils de préférences plus ou moins concentrés autour d'un consensus central.

Le modèle de Mallows est défini par deux paramètres principaux :

- Un classement central de référence  $P_0 \in \mathcal{L}(\mathcal{C})$ .
- Un paramètre de dispersion  $\theta \geq 0$ .

Contrairement à un modèle uniforme où chaque classement aurait la même probabilité d'être choisi par un votant, le modèle de Mallows attribue une probabilité décroissante aux classements à mesure qu'ils s'éloignent de  $P_0$ . La probabilité d'observer un classement  $\sigma$  est définie par :

$$P(\sigma|P_0, \theta) = \frac{\exp(-\theta \cdot D_\tau(\sigma, P_0))}{\psi(\theta)}$$

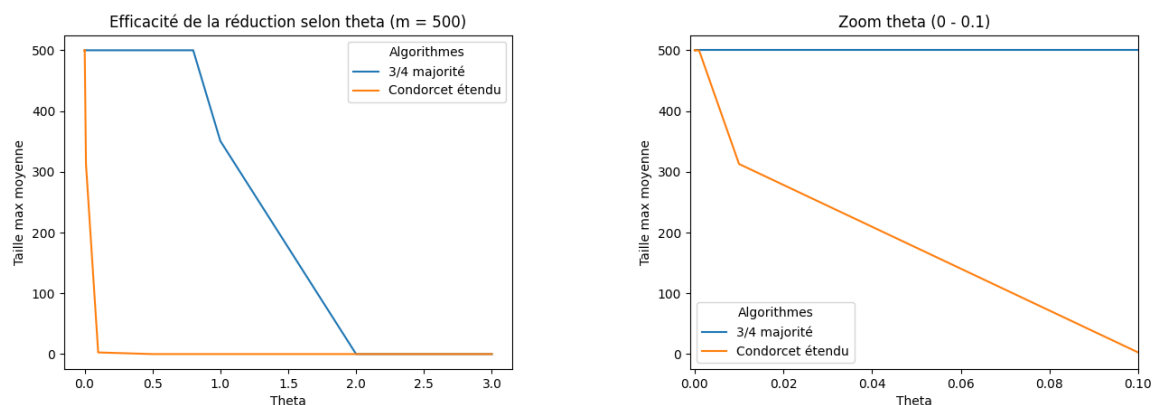
où  $\psi(\theta)$  est la fonction de normalisation garantissant que  $\sum_{\sigma \in \mathcal{L}(\mathcal{C})} P(\sigma) = 1$ .

L'intérêt de ce modèle réside dans la manipulation de  $\theta$  :

- Si  $\theta$  est proche de 0, le modèle tend vers une distribution uniforme. Les préférences sont alors très hétérogènes, ce qui rend la recherche du classement de Kemeny et la décomposition de l'instance plus complexes.
- À l'inverse, plus  $\theta$  est élevé, plus la probabilité d'un vote décroît rapidement avec sa distance à  $P_0$ , concentrant l'opinion autour du consensus.

Pour nos expérimentations, nous avons utilisé l'implémentation issue du dépôt GitHub <https://github.com/ekhiru/top-k-mallows.git>. Ce code permet de générer des profils de préférences suivant le modèle de Mallows. Nous avons ainsi pu générer des instances en faisant varier indépendamment le nombre de candidats  $m$  et le paramètre de dispersion  $\theta$  nous permettant d'analyser les performances de nos algorithmes.

### 5.2.1 Comparaison des méthodes de décomposition



(a) Vue d'ensemble sur l'intervalle  $\theta \in [0, 3]$ .

(b) Zoom sur le comportement critique.

FIGURE 3 – Efficacité de la réduction de taille des instances selon theta.

Nous avons généré 50 instances de  $m$  candidats avec le même theta. Pour chacune d'elles, nous avons évalué la taille de la plus grande sous-instance. Les courbes représentent la taille moyenne de la plus grande sous-instance sur les instances générées.

De nouveau, on observe que le critère de Condorcet étendu réduit d'avantage que la règle de majorité des 3/4, quelque soit theta.

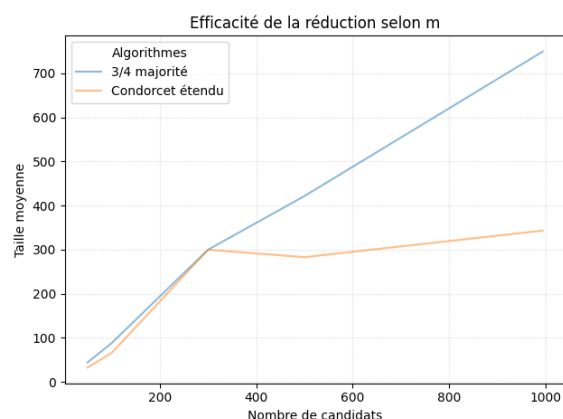


FIGURE 4 – Efficacité de la réduction de taille des instances selon le nombre de candidats.

### 5.2.2 Evaluation de la programmation linéaire

## 6 Conclusion

## Références

- [1] Dwork Cynthia, Kumar Ravi, Moni Naor, and D. Sivakumar. Rank aggregation methods for the web. 2001.
- [2] Davenport Andrew and Kalagnanam Jayant. A computational study of the kemeny rule for preference aggregation. 2004.
- [3] Young H. P. and Levenglick A. A consistent extension of condorcet's election principle. 1978.

- [4] Nadja Betzler, Robert Brederbeck, and Rolf Niedermeier. Theoretical and empirical evaluation of data reduction for exact kemeny rank aggregation. 2013.
- [5] Fischer Felix, Hudry Olivier, and Niedermeier Rolf. Weighted tournament solutions. 2016.
- [6] Truchon Michel. An extension of the condorcet criterion and kemeny orders. Technical report, 1999.
- [7] Mattei and Walsh. Preflib : A library for preferences. 2013.
- [8] Tarjan Robert. Depth-first search and linear graph algorithms. 1972.
- [9] Nadja Betzler, Michael R. Fellows, Jiong Guo, Rolf Niedermeier, and Frances A. Rosamond. Fixed-parameter algorithms for kemeny rankings. 2009.
- [10] Mallows. Non-null ranking models. 1957.